Lab 2

Course Title: Data Mining

Course Code: CSE 477

Semester: Spring 2026

Section: 01

Submitted by

Name: Shafiqul Islam Fahim

ID: 2022-2-60-085

Date: 25/1/2026

Submitted to

Amit Mandal

Lecturer, Department of Computer Science and Engineering

East West University

1.Introduction

In this week's lab, titled "Data Cleaning & Preprocessing for YouTube Text Mining," I worked with the YouTube dataset created during the previous lab on exploratory data analysis. This lab focused on transforming the raw text data into a format suitable for Natural Language Processing (NLP) tasks, rather than concentrating on visualization or engagement patterns. The dataset included YouTube comments and auto-generated captions, both of which contained noise such as usernames, timestamps, inconsistent spacing, special symbols, and repeated words. To address these issues, I utilized NLTK and other Python libraries to construct a structured preprocessing pipeline. This approach, in contrast to manual cleaning techniques, provided valuable insights into how NLP-based preprocessing can yield cleaner, more meaningful tokens and enhance the overall quality of text mining datasets.

2. Objective

The objective of this lab is to clean and preprocess raw textual data from YouTube comments and captions. The primary goal is to convert unstructured, noisy text data into a structured format that can be used for further analysis or text mining tasks. This lab focuses on removing irrelevant content, handling inconsistencies, and transforming the data into a standardized form, making it ready for analysis in subsequent labs.

- **Key Goals**:
  - Convert raw data from `.txt` (comments) and `.vtt` (captions) files into a structured format.
  - Apply various data cleaning techniques like text normalization, tokenization, stopword removal, and lemmatization.
  - Save the cleaned data for future use in text mining projects.

3. Tools and Libraries Used

Python Version:

- Python 3.12 (or the specific version you are using).

Libraries:

- **pandas**: Used for handling and analyzing data in tabular form. It helps load, clean, and manipulate large datasets efficiently.
- **matplotlib**: Used for visualizing data (e.g., bar charts, word clouds) to gain insights from the cleaned text.
- **nltk (Natural Language Toolkit)**: A key library for text preprocessing. It provides tools for tokenization, stopword removal, stemming, and lemmatization.
- **webvtt-py**: A library to read and parse `.vtt` files containing YouTube captions.
- **re**: For handling regular expressions (used in cleaning text data).
- **string**: To handle common string operations, such as punctuation removal.

Python Environment:

- Colab, Jupyter Notebook, or your local Python environment (mention the environment you used for running the lab).

4. Dataset Description

- **comments.txt**:
  - Contains raw YouTube comment data. Each line includes:
    - **Username**: The name of the user who posted the comment.
    - **Timestamp**: The time at which the comment was posted.
    - **Comment Text**: The actual comment made by the user.
  - **Issues/Irregularities**:
    - **Blank lines**: Some lines may be empty and need to be ignored during processing.
    - **Replies**: Some comments may be replies to other comments (e.g., text like "Reply to comment #123").
    - **Mixed languages**: Some comments may contain non-English text, which could create inconsistencies.
- **captions.vtt**:
  - Contains subtitle data for a YouTube video, with each caption having:
    - **Timestamp**: When the caption appears in the video.
    - **Caption Text**: The text shown in the subtitle at the given time.
  - **Issues/Irregularities**:
    - **Inconsistent formatting**: Some captions may have incorrect or missing timestamps.
    - **Empty captions**: Some caption entries might not contain any text.
    - **Non-English text**: Some captions may be in different languages or have special characters.

5. Methodology / Procedure

5.1 Profiling the Data

At the beginning of this lab, I first inspected the raw datasets to understand how the textwas stored and what kind of irregularities existed. The comment dataset(all_comments.txt) was not organized in a table format; instead, every comment wasstored as a single line that contained multiple pieces of information such as the commentnumber, username, timestamp, likes, and the comment message. While checking the filemanually, I noticed that some lines contained extra space, special symbols, emojis, andsometimes HTML-like entities such as &. These issues can create problems duringtext mining if they are not cleaned properly.Similarly, I inspected the caption dataset (captions.en.vtt) which was in WebVTT format.This file contained timestamps and subtitle text cues, but it also included blank lines andformatting tags (such as <c> and embedded timestamp markers). Another importantissue in the caption file was that auto-generated captions often repeat the last few wordsfrom the previous cue at the beginning of the next cue. This overlap creates duplicate wordsand affects

sentence clarity. Profiling the datasets helped me understand these problemsearly so that I could design the parser and clean pipeline more accurately

5.2 Parsing Comments and Captions

After understanding the raw structure, I converted both datasets into structured formatsusing Pandas DataFrames so that the text could be processed efficiently.For parsing the comments, I used a regular expression pattern to extract important fieldsfrom each comment line. From every valid line, the parser collected the username,timestamp, number of likes, and the comment text. All extracted values were stored into a5DataFrame with columns like username, timestamp, likes, and comment_text. This stepwas necessary because raw comment text alone is not enough; metadata like timestampand likes are also useful for future analysis.For parsing captions, I used the webvtt-py library to read subtitle cues from the .vtt file.Each cue contains a start time, end time, and the caption text. Since captions ofteninclude repeated words between consecutive cues, I applied an overlap-removal techniqueto merge captions smoothly while avoiding duplication. After that, the combined captiontext was split into meaningful sentences using sentence-splitting logic. Finally, captionswere structured into a DataFrame containing Start_time, End_time, andcaption_sentence, which made the dataset ready for preprocessing.

5.3 Cleaning and Preprocessing Pipeline

- **Normalization**: Convert all text to lowercase to maintain uniformity. Remove punctuation and special characters to avoid noise in the data.
- **Tokenization**: Split the text into individual words (tokens) using the `nltk.word_tokenize` function.
- **Stopword Removal**: Filter out common, uninformative words like "the", "is", "in", etc., using `nltk.corpus.stopwords`.
- **Lemmatization**: Reduce words to their root form using `nltk.WordNetLemmatizer` to ensure words like “running” and “ran” are treated as the same word “run”.

Example Code for Cleaning:

```
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer
import string

def clean_text(text):
    text = text.lower()
    text = text.translate(str.maketrans("", "", string.punctuation))
    tokens = word_tokenize(text)
    stop_words = set(stopwords.words("english"))
    tokens = [word for word in tokens if word not in stop_words]
    lemmatizer = WordNetLemmatizer()
    tokens = [lemmatizer.lemmatize(word) for word in tokens]
    return " ".join(tokens)
```

5.4 Exporting Cleaned Data

- After cleaning, export the cleaned data to CSV files for future analysis:
    - **cleaned_comments.csv**: Contains the cleaned YouTube comments data.

- **cleaned_captions.csv**: Contains the cleaned YouTube captions data.

Example Code:

```
def process_and_export_data(comment_file, caption_file):
    comments_df = parse_comments(comment_file)
    cleaned_comments = comments_df["comment_text"].apply(clean_text)
    captions = parse_captions(caption_file)
    cleaned_captions = [clean_text(caption) for caption in captions]

    comments_df["cleaned_comment"] = cleaned_comments
    pd.DataFrame(cleaned_captions,
columns=["cleaned_caption"]).to_csv("cleaned_captions.csv",
index=False)
    comments_df.to_csv("cleaned_comments.csv", index=False)
```

6. Code Implementation

Cleaning Function :

```
# Cleaning pipeline
stop_words = set(stopwords.words('english'))
lemmatizer, stemmer = WordNetLemmatizer(), PorterStemmer()

def normalize_text(text: str) -> str:
    if text is None: return ""
    text = text.lower()
    text = re.sub(r'\[.*?]', ' ', text)
    text = re.sub(r'[^a-z\s]', ' ', text)      # change to [^a-z0-9\s] to
keep digits
    text = re.sub(r'\s+', ' ', text).strip()
    return text

def tokenize_text(text: str):
    if not text: return []
    return word_tokenize(text)

def remove_stopwords(tokens):
    return [w for w in tokens if w not in stop_words and len(w) > 2]

def lemmatize_tokens(tokens):
    return [lemmatizer.lemmatize(w) for w in tokens]

def stem_tokens(tokens):
    return [stemmer.stem(w) for w in tokens]

def clean_text_pipeline(text: str, use_lemmatization: bool = True):
    x = normalize_text(text)
    toks = tokenize_text(x)
    toks = remove_stopwords(toks)
    return lemmatize_tokens(toks) if use_lemmatization else
stem_tokens(toks)
```

```
# Comments parser
def structure_comments_from_txt(filepath='all_comments.txt'):
    pat = re.compile(
        r'^\s*\d+\.\s*@([\w\-]+)\s*\[([0-9T:\-
Z]+)\]\s*([\d,]+)\s*Likes?\s*\u2192\s*(.+)$'
    )
    rows = []
    with open(filepath, 'r', encoding='utf-8') as f:
        for raw in f:
            m = pat.match(raw.strip())
            if not m:
                continue
            username  = m.group(1)
            timestamp = m.group(2)
            likes = int(m.group(3).replace(',', ''))
            comment = m.group(4).strip()
            rows.append({'username': username,
                         'timestamp': timestamp,
                         'likes': likes,
                         'comment_text': comment})
    return pd.DataFrame(rows,
columns=['username','timestamp','likes','comment_text'])

# Captions parser
_SENT_SPLIT = re.compile(r'(?<=[.!?])\s+')

def _c(s: str) -> str:
    # minimal clean: strip, collapse spaces, drop stray quotes
    s = ' '.join(s.strip().split())
    # Corrected the string literal to include all desired quote characters
for stripping
    return s.strip('"\'“”‘’')

def _deoverlap(prev_line: str, curr_line: str, k: int = 6) -> str:
    """
    Remove up to k-word overlap between the end of prev_line and start of
curr_line.
    This kills the rolling-window repetition common in auto-captions.
    """
    a = prev_line.split()
    b = curr_line.split()
    for n in range(min(k, len(a), len(b)), 0, -1):
        if a[-n:] == b[:n]:
            return ' '.join(b[n:])
    return curr_line

def structure_captions_from_vtt(filepath='captions.en.vtt'):
    # read all cues (assumes webvtt works; no extra error handling)
```

```
    cues = [_c(cue.text) for cue in WebVTT().read(filepath) if
cue.text.strip()]
    if not cues:
        return pd.DataFrame({'caption_sentence': []})

    # merge cues while removing small overlaps
    merged = [cues[0]]
    for t in cues[1:]:
        extra = _deoverlap(merged[-1], t, k=6)
        if extra:
            merged[-1] = merged[-1] + ' ' + extra

    # split into sentences
    big_text = ' '.join(merged)
    sentences = [s.strip() for s in _SENT_SPLIT.split(big_text) if
s.strip()]

    return pd.DataFrame({'caption_sentence': sentences})

# Preview
comments_df = structure_comments_from_txt('all_comments.txt')
comments_df['cleaned_tokens'] =
comments_df['comment_text'].apply(clean_text_pipeline)
print("Comments head:")
print(comments_df.head())

captions_df = structure_captions_from_vtt('captions.en.vtt')
captions_df['cleaned_tokens'] =
captions_df['caption_sentence'].apply(clean_text_pipeline)
print("\nCaptions head:")
print(captions_df.head(20))
```

7. Results and Discussion

Data Overview

- **Comments**: 216 rows
- **Captions**: 574 rows

Tokenization Examples

To better understand the effect of preprocessing, I examined a few sample lines from both comments and captions. The raw text was tokenized into meaningful tokens after cleaning.

- **Example 1 (Comment)**:

    - **Raw Text**: "post quantum polymorphic prompt injection via deep fake <a href='http://www.youtube.com/results?search_query=%23ai'>#ai</a>"
    - **Cleaned Tokens**: `['post', 'quantum', 'polymorphic', 'prompt', 'injection', 'via', 'deep', 'fake', 'href', 'http', 'www', 'youtube']`
- **Example 2 (Comment)**:
    - **Raw Text**: "absolute perfection, what a tutorial!"
    - **Cleaned Tokens**: `['absolute', 'perfection', 'tutorial']`
- **Example 1 (Caption)**:
    - **Raw Text**: "it's become something of a tradition here on the IBM technology channel to do here on the IBM technology channel to do h"
    - **Cleaned Tokens**: `['become', 'something', 'tradition', 'ibm', 'technology', 'channel', 'ibm', 'technology', 'channel', 'ibm', 'technology', 'channel']`
- **Example 2 (Caption)**:
    - **Raw Text**: "we did this in 2023, for the future."
    - **Cleaned Tokens**: `['future']`

These examples show how the raw text is transformed into a list of meaningful tokens after cleaning, with unnecessary words and symbols removed.

Stopword Removal

A significant part of the preprocessing was the removal of stopwords. In total, 7352 stopwords were removed, with:

- **Comments**: 1978 stopwords
- **Captions**: 5374 stopwords

Removing stopwords helped eliminate common filler words like “the”, “is”, “and”, which allowed the dataset to focus on more informative words, thus improving the overall quality of the tokenized data.

Frequent Tokens

After preprocessing, I calculated the most frequent tokens in both datasets.

- **Top Tokens in Comments (10 most frequent)**:
    1. 'video' (27 occurrences)
    2. 'quantum' (26 occurrences)
    3. 'shadow' (18 occurrences)
    4. 'like' (18 occurrences)
    5. 'tech' (16 occurrences)
    6. 'deepfakes' (16 occurrences)
    7. 'use' (16 occurrences)
    8. 'sound' (16 occurrences)
    9. 'would' (15 occurrences)
    10. 'kinda' (15 occurrences)

These tokens indicate that the most common topics in comments relate to video content, quantum technology, and AI, particularly deepfakes and tech-related discussions.

- **Top Tokens in Captions (10 most frequent)**:
    1. 'going' (139 occurrences)
    2. 'agent' (79 occurrences)
    3. 'see' (72 occurrences)
    4. 'thing' (67 occurrences)
    5. 'one' (55 occurrences)
    6. 'think' (43 occurrences)
    7. 'well' (41 occurrences)
    8. 'attack' (36 occurrences)
    9. 'would' (34 occurrences)
    10. 'seen' (32 occurrences)

In captions, the frequent tokens reflect a more explanatory and instructional tone, focusing on words related to agents, things, and predictions.

Data Quality Check

A data quality check revealed the following issues:

- **Comments**:
    1. Non-ASCII characters: 51 occurrences
    2. Special symbols: 53 occurrences
- **Captions**:
    1. Non-ASCII characters: 0 occurrences
    2. Special symbols: 5 occurrences

These issues highlight that real-world YouTube data is noisy, and preprocessing is essential for cleaning non-ASCII characters, special symbols, and other irregularities from the dataset.

Engagement Analysis

To explore whether longer comments receive more engagement, I examined the correlation between likes and different length features of the comments. The correlations were as follows:

- **Likes and Word Length**: 0.057
- **Likes and Token Length**: 0.053
- **Likes and Character Length**: 0.057

The correlations were weak but slightly positive, suggesting that longer comments do not necessarily receive more likes. This implies that engagement is likely driven by the quality, relevance, and uniqueness of the comments, rather than their length.

Most-Liked Comments

The top three most-liked comments in the dataset were:

1. **más-g7z** — 139 likes: "I have a feeling that companies are going to realize that some AI implementation"
2. **Bhanupratap-w7j** — 107 likes: "IBM is mostly spot on with the trends predictions"
3. **Howell_Jolly** — 77 likes: "After signing in with a passkey or a physical security key for multi-factor auth"

These highly liked comments indicate that users engage with content that is insightful, reflects positive sentiments, or addresses trending topics in a supportive manner.

8. Conclusion

This analysis shows how raw YouTube comments and captions were effectively preprocessed into structured and meaningful datasets. The cleaning pipeline—including tokenization, stopword removal, and engagement analysis—helped extract useful patterns and insights. The weak correlations between comment length and likes suggest that engagement is influenced more by content relevance and quality than by the length of the comment. This cleaned and tokenized data can be used for more advanced tasks such as sentiment analysis, keyword extraction, and topic modeling, providing valuable insights into user engagement and content discussions.T

9. References

NLTK
DocumentationPandas Documentation
Python re Module Documentation
webvtt-py Documentation